Samadhan Setu / VeriCity

Civic Governance & Proof-of-Work Blockchain Verification Platform

Document: System Features & Platform Integration Matrix · **Target Ecosystem:** Web3, Embedded Wallets, IPFS, Cloud & UI · **Version:** 2.4 Production Specification · **Blockchain:** Polygon Amoy Testnet (Chain ID 80002)

EXECUTIVE ARCHITECTURAL SUMMARY:
Samadhan Setu leverages a high-throughput hybrid Web2-Web3 civic architecture designed for zero citizen friction, gasless mobile participation, and mathematical auditability on a public blockchain. A foundational engineering design in this project is the strict separation of **Citizen Gasless Cryptographic Signing** from **Municipal Authority Smart Contract Relaying**. This document details how **MetaMask and Privy** operate synergistically, followed by the comprehensive 4-section matrix detailing the **Feature**, **Platform**, **Use (Role & Purpose)**, and **Flow (Execution Architecture)** across the entire system.

1. The Core Wallet Architecture: Did We Use MetaMask or Privy?

Direct Architectural Answer: We use **BOTH MetaMask and Privy** in Samadhan Setu. Neither is redundant; they serve entirely different architectural tiers, user personas, and transaction models:

• **Privy (Citizen Identity & Gasless Signing Layer):** Everyday citizens cannot be expected to install browser extensions, manage 12-word seed phrases, or buy MATIC/POL cryptocurrency to report a broken pothole or street light. Privy provisions *embedded, non-custodial Ethereum wallets* generated instantaneously via mobile phone OTP. Citizens digitally sign civic reports silently in the background (EIP-191) with **zero gas fees and zero transaction popups**.
• **MetaMask (Municipal Deployer & Automated Relayer Layer):** MetaMask is the administrative Web3 operational wallet. It connects to Polygon Amoy Testnet (Chain ID 80002) and holds test POL tokens. Developers and authorities use MetaMask to deploy the core Solidity contracts (ResolutionAnchor.sol and OfficerReputation.sol) and fund the automated nightly Node.js cron relayer (cron-anchor.mjs) that anchors Merkle roots and commits officer reputation scores to the public blockchain.

Dimension / Criteria	Privy (Citizen Embedded Wallet)	MetaMask (Municipal & Relayer Wallet)
Target User & Persona	Everyday public citizens reporting civic defects & verifying repairs	Municipal administrators, smart contract deployers & backend cron relayer
User Interface & UX	Seamlessly embedded in Next.js; invisible; zero extensions; zero confirmation popups	Browser extension / hardware wallet; explicit popup approvals for transactions
Onboarding & Auth	Mobile Phone OTP (SMS / Supabase Auth) + deterministic SHA-256 fallback	BIP-39 12/24-word recovery seed phrase; password protected
Gas Fees & Tokens	**100% Gasless for citizen.** Citizen holds 0 POL; never spends cryptocurrency	**Gas-bearing.** Holds POL/MATIC on Polygon Amoy to pay on-chain transaction gas
Cryptographic Output	Off-chain EIP-191 digital signatures verifying complaint authenticity & timestamp	On-chain EVM transactions: contract deployment, state mutations, Merkle root commits
Key Enclave & Storage	Privy client-side iframe / MPC secure enclave (or local PBKDF2 salt in demo mode)	Encrypted browser vault; deployer private key exported to backend .env
Codebase Location	frontend/samadhan-setu/lib/privy.ts, PrivyProviderWrapper.tsx	blockchain/hardhat.config.cjs, backend/scripts/cron-anchor.mjs

Wallet Execution Architecture: Privy Flow vs. MetaMask Flow

PRIVY CITIZEN SIGNING FLOW (CLIENT-SIDE):
1. Citizen navigates to /citizen and enters mobile phone number.
2. OTP verified via Supabase/Privy; Privy generates embedded Ethereum address (0x...).
3. Address mapped into Supabase citizen_wallets table.
4. App constructs canonical string: --- VERICITY REPORT --- Domain | Location | Time.
5. privySigner.signMessage() silently generates EIP-191 signature without gas fees.
6. Edge function cryptographically recovers signer address to verify authenticity before insert.

METAMASK MUNICIPAL RELAYER FLOW (ADMIN & BACKEND):
1. Admin connects MetaMask to Polygon Amoy Testnet (Chain ID 80002) and funds with test POL.
2. Deployer private key exported to Hardhat and backend .env configs.
3. Hardhat compiles and deploys ResolutionAnchor.sol & OfficerReputation.sol.
4. Nightly cron (cron-anchor.mjs) loads ethers.Wallet(DEPLOYER_KEY, amoyProvider).
5. Relayer calls anchorMerkleRoot(root, count) and recordScoresBatch().
6. Emits verified events on Polygonscan; QR codes in app link directly to public transactions.

2. Master Architecture Matrix: Features, Platforms, Uses & Flows

Structured 4-section table covering system capabilities, platforms, functional roles, and step-by-step execution flows:

Feature	Platform Used	Use (Role & Purpose)	Flow (Step-by-Step Architecture)
Citizen Embedded Wallet & Identity	**Privy @privy-io/react-auth + Deterministic Key**	Frictionless Web3 onboarding for everyday citizens without browser extensions or seed phrases. Creates persistent 0x... address tied to phone.	**1.** Phone input -> **2.** SMS/Supabase OTP verify -> **3.** Privy derives Ethereum keypair -> **4.** Address stored in citizen_wallets -> **5.** Session key stored in client enclave.
Civic Complaint Digital Signing	**Privy Signer (EIP-191 Personal Sign) + Ethers.js Wallet**	Cryptographic proof of origin and digital non-repudiation. Guarantees complaints cannot be forged or altered by rogue admins.	**1.** Citizen inputs details & photo -> **2.** Client constructs canonical signing string -> **3.** Privy silently signs message -> **4.** Signature passed to Edge Function -> **5.** Function validates proof before DB commit.
Smart Contract Deployment & Admin	**MetaMask + Hardhat CLI + Polygon Amoy**	Municipal administrative control and smart contract lifecycle management. Holds test POL to fund EVM contract deployment and relayer operations.	**1.** Admin installs MetaMask on Amoy (80002) -> **2.** Acquires test POL -> **3.** Exports private key to Hardhat .env -> **4.** Deploys contracts -> **5.** Addresses saved to deployed-contracts.json.
Batch Merkle Root Anchoring	**Ethers.js Relayer (Funded via MetaMask) + ResolutionAnchor.sol**	Scalable on-chain proof-of-work. Bundles hundreds of daily repair records into a single Merkle Root on Polygon Amoy, slashing gas by 99.9%.	**1.** Nightly cron reads resolved cases -> **2.** Computes SHA-256 record hashes -> **3.** Builds binary Merkle tree -> **4.** Relayer sends anchorMerkleRoot() -> **5.** Transaction emits public Polygonscan event.
Officer On-Chain Reputation Tracking	**Polygon Amoy + OfficerReputation.sol + Edge Functions**	Transparent, immutable merit scoring for field officers. Eliminates favoritism by recording performance basis points (0-10000) on a public ledger.	**1.** Citizen confirms (+2) or disputes (-6) repair -> **2.** Edge function recalculates score -> **3.** Nightly cron gathers scores -> **4.** Invokes recordScoresBatch() -> **5.** Stores immutable history on-chain.
Decentralized Photo Evidence Storage	**Pinata IPFS (InterPlanetary File System) API + Gateway**	Permanent, content-addressed decentralized storage for post-repair 'after' photos taken by field officers. Prevents retroactive tampering.	**1.** Officer snaps after-repair photo -> **2.** uploadToPinata() pins image to IPFS -> **3.** Pinata returns CIDv1 (bafy...) -> **4.** CID saved in resolution record -> **5.** CID included in Merkle leaf hash.

2. Master Architecture Matrix (Continued)

Feature	Platform Used	Use (Role & Purpose)	Flow (Step-by-Step Architecture)
Live Camera Capture & Watermarking	**HTML5 MediaStream navigator.mediaDevices + HTML5 Canvas 2D**	Anti-fraud photo capture engine. Disables pre-existing gallery uploads and burns forensic timestamp & GPS watermark directly into raw pixels.	**1.** User taps capture -> **2.** Browser streams environment camera -> **3.** Video frame drawn to Canvas -> **4.** Live GPS & time burned onto pixels -> **5.** Strips EXIF; outputs clean un-spoofed JPEG.
Forensic GPS Geofencing	**Browser Geolocation + Supabase PostGIS / Edge Functions**	Spatial verification: guarantees complaints and repair evidence are captured directly at the physical defect location. Rejects imprecise GPS (> 50m).	**1.** Device queries high-accuracy GPS -> **2.** Validates accuracy <= 50m -> **3.** Lat/long locked to camera capture -> **4.** Edge Function runs validLocation() -> **5.** Rejects out-of-bounds tickets.
Whistleblower Pseudonymity (HMAC)	**Web Crypto API + Supabase Functions (HMAC-SHA256)**	Protects citizens from harassment by municipal officers by generating a deterministic pseudonymous identifier (derived_citizen_id).	**1.** Citizen logs in with phone -> **2.** Edge function runs: HMAC-SHA256(secret, user_id + complaint_id) -> **3.** Generates derived hash -> **4.** Displays on public portals; true phone kept secret.
Spatial Duplicate Detection & Upvotes	**Supabase PostgreSQL (Haversine RPC nearby_complaints)**	Clutter prevention and community validation. Identifies existing complaints within 75m; lets citizens upvote and add proof instead of duplicate tickets.	**1.** Citizen chooses domain & gets GPS -> **2.** Queries /nearby API -> **3.** Calculates geodetic distance <= 75m -> **4.** Suggests upvote -> **5.** Upvote increments counter & escalates SLA priority.
Role-Based Governance Workflow	**Next.js 14 App Router + Tailwind CSS + Supabase RLS**	3-tier operational governance separating Citizen (/citizen), Municipal Authority (/authority), and Field Officer (/officer) portals.	**1.** Citizen files (filed) -> **2.** Authority assigns officer (assigned) -> **3.** Officer repairs defect (in_progress) -> **4.** Officer uploads IPFS proof -> **5.** Citizen verifies -> resolved.
Public Transparency & QR Code Explorer	**qrcode Library + Polygonscan API (Amoy Explorer)**	Zero-barrier public auditability. Allows citizens and civic watchdogs to scan a QR code to inspect smart contract events, Merkle roots, and scores on-chain.	**1.** User taps 'Verify On-Chain' [Proof] -> **2.** Modal reads contract address & txHash -> **3.** Generates dynamic QR Code -> **4.** User scans with phone -> **5.** Polygonscan displays verified block & parameters.

SYSTEM COMPONENT SYNERGY: HOW THE PLATFORMS INTERCONNECT:

- **Frontend (Next.js + Privy):** Authenticates citizen via Phone OTP, accesses hardware camera/GPS, and signs ticket via Privy EIP-191.
- **Backend (Supabase Edge Functions + PostgreSQL):** Validates signature, computes HMAC pseudonym, stores metadata, and enforces RLS.
- **Storage Layer (Pinata IPFS):** Content-addresses 'after-repair' photos, returning CIDv1 hashes for tamper-proof evidence.
- **Relayer Engine (MetaMask / Ethers.js Cron):** Aggregates day's resolution hashes into binary Merkle Tree and pays gas to commit on Polygon Amoy.
- **Public Verification (Polygonscan + QR):** Exposes smart contract events for independent external auditability by any citizen or journalist.

3. Technical Stack & Infrastructure Inventory

Layer / Domain	Technology / Framework	Version / Specification	Architectural Responsibility
Frontend Framework	**Next.js (App Router)**	v14.2+ / TypeScript 5.4+	Server & Client Components, Responsive Civic Portals, Tailwind CSS
Citizen Web3 Layer	**Privy React SDK**	@privy-io/react-auth	Gasless embedded Ethereum wallets, phone OTP linking, silent EIP-191 signing
Admin Blockchain	**MetaMask & Hardhat**	Hardhat Toolbox / Solidity 0.8.20	Contract compilation, Polygon Amoy deployment, deployer relayer private key
Public Blockchain	**Polygon Amoy Testnet**	Chain ID: 80002 / EVM	Decentralized state anchor: ResolutionAnchor & OfficerReputation
Decentralized Storage	**Pinata IPFS**	IPFS Pinning API v1 / CIDv1	Content-addressed decentralized permanent storage of post-repair evidence
Backend Cloud & DB	**Supabase**	PostgreSQL 15 / Edge Functions	Row Level Security, Geodetic spatial queries, HMAC identity, Storage buckets
Media & Capture	**HTML5 MediaStream & Canvas**	Web API Standard	Live environment camera streaming, anti-spoofing, pixel-level watermarking
Cryptographic Tools	**Ethers.js & Web Crypto**	Ethers v6.11 / SHA-256 / Keccak	Binary Merkle tree construction, signature verification, deterministic derivation

4. End-to-End Civic Complaint Lifecycle Walkthrough

STAGE 1: CITIZEN REPORTING & PRIVY SIGNING

- Citizen visits /citizen, verifies phone via OTP, and receives an embedded Privy Ethereum address (0x...).
- Citizen captures live photo via HTML5 camera module; system embeds forensic GPS coordinates & timestamp into canvas.
- System checks for duplicates within 75m via Haversine RPC; Privy silently signs complaint via EIP-191 personal sign.
- Supabase Edge Function validates signature, computes HMAC pseudonym (derived_citizen_id), and sets status to 'filed'.

STAGE 2: MUNICIPAL TRIAGE & OFFICER ASSIGNMENT

- Municipal Authority logs into /authority, reviews verified GPS coordinates, domain category, and before-photo proof.
- Authority assigns complaint to qualified domain officer with an enforced SLA resolution deadline (status -> 'assigned').

STAGE 3: FIELD REPAIR & ON-SITE EXECUTION

- Field Officer logs into /officer and accepts case (status -> 'in_progress'); arrives at physical scene to execute repair.
- Officer captures 'after-repair' photo using live camera module with verified GPS confirmation ensuring on-site presence.

STAGE 4: PINATA IPFS UPLOAD & RECORD HASHING

- Photo is uploaded to Pinata IPFS; returns permanent CIDv1 (bafy...).
- System computes cryptographic hash: record_hash = sha256(complaintId | officerId | photoHash | lat | long | prevHash).

STAGE 5: MUNICIPAL APPROVAL & CITIZEN 2-PARTY VERIFICATION

- Supervisor reviews officer's IPFS proof and approves resolution; citizen receives prompt to confirm or dispute site repair.
- Citizen confirmation adjusts officer merit score (+2 confirmed, -6 disputed); complaint status advances to 'resolved'.

STAGE 6: NIGHTLY MERKLE ANCHORING VIA METAMASK RELAYER

- Backend cron (cron-anchor.mjs) pulls day's record hashes, builds binary Merkle Tree, and extracts 32-byte root hash.
- Relayer wallet (funded via MetaMask) submits anchorMerkleRoot() and recordScoresBatch() to Polygon Amoy.
- Polygon emits immutable events; QR codes in frontend allow instant zero-knowledge verification on Polygonscan.